Activity No. 6 Introduction to Core APIs	
Course Code: CPE 025B	**Program:** Computer Engineering
Course Title: Software Design 2	**Date Performed: 04/01/26**
Section: CPE32S1	**Date Submitted: 04/01/26**
Name: Edson Ray San Juan	**Instructor: Engr. Neil Matira**

1. Objective(s)

This activity aims to introduce more APIs to students that can be used in developing a simple messaging application in React Native.

2. Intended Learning Outcomes (ILOS)

After this module, the student should be able to:

- Create cross-platform interactions for deleting and enlarging messages by utilizing core APIs in React Native.
- Demonstrate the use of React Native in mobile app development using additional core APIs.

3. Materials and Equipment

To properly perform this activity, the student must have:

- Node.js LTS
- Visual Studio Code
- React Native
- Android Emulator/Simulator

4. Procedure & Discussion

<u>The Message Utils</u>

Let's first write a few utility functions for creating message objects so that we keep this logic separate from our rendering logic. Create a new directory called utils in the messaging directory. Within utils, create a new file called MessageUtils.js.

Within Message Utils.js, let's first define the shape of each message using PropTypes.shape. All of the messages we render will have a type and an id, and then some messages will have either a text, uri, or coordinate value, depending on the type.

Add the following to MessageUtils.js:

```javascript
import PropTypes from 'prop-types';

export const MessageShape = PropTypes.shape({
  id: PropTypes.number.isRequired,
  type: PropTypes.oneOf(['text', 'image', 'location']),
  text: PropTypes.string,
  uri: PropTypes.string,
```

```
  coordinate: PropTypes.shape({
    latitude: PropTypes.number.isRequired,
    longitude: PropTypes.number.isRequired,
  }),
});
```

By using this shape in the propTypes of a component, React will automatically warn us if we accidentally pass invalid message data.

Question: What is the benefit of declaring models this way?

Let's write a few utility functions for creating the different kinds of messages:

```javascript
let messageId = 0;

function getNextId() {
    messageId += 1;
    return messageId;
}

export function createTextMessage(text) {
    return {
        type: 'text',
        id: getNextId(),
        text,
    };
}

export function createImageMessage(uri) {
    return {
        type: 'image',
        id: getNextId(),
        uri,
    };
}

export function createLocationMessage(coordinate) {
    return {
        type: 'location',
        id: getNextId(),
```

```
        coordinate,
    };
}
```

We created a utility getNextId() for getting a unique message id. It's important that we ensure uniqueness for each id, since we'll be using the id as the key when rendering these messages in a list.

Question: Why is it important for us to ensure the uniqueness of each id?

By exporting createTextMessage, createImageMessage, and createLocationMessage, we can now easily create new messages of each type from elsewhere in our app. We'll use these messages to populate the FlatList.

MessageList

Our MessageList component will render an array of the message objects we defined in Message Utils. We can determine how to render each message based on its type. Let's start by determining the propTypes for this component. Here's a good opportunity to use the MessageShape we just defined. We'll also want to notify the parent component whenever a message in the list is pressed. We can do this using an onPressMessage function prop.

Create a new file, MessageList.js, in our components directory. Add the following to it:

messaging/components/MessageList.js

```javascript
import React from 'react';
import PropTypes from 'prop-types';
import { MessageShape } from '../utils/MessageUtils';

export default class MessageList extends React.Component {
  static propTypes = {
    messages: PropTypes.arrayOf(MessageShape).isRequired,
    onPressMessage: PropTypes.func,
  };

  static defaultProps = {
    onPressMessage: () => {},
  };
}
```

By using our MessageShape, React will warn us if we're passed malformed data. Now let's render our

messages into a FlatList. Just as in the previous chapter, We need to use a keyExtractor to tell the FlatList how to find the unique id of our message objects. Let's update MessageList.js to render a FlatList:

```javascript
import React from 'react';
import PropTypes from 'prop-types';
import { FlatList, StyleSheet } from 'react-native';
import { MessageShape } from '../utils/MessageUtils';

const keyExtractor = item => item.id.toString();

export default class MessageList extends React.Component {
  static propTypes = {
    messages: PropTypes.arrayOf(MessageShape).isRequired,
    onPressMessage: PropTypes.func,
  };

  static defaultProps = {
    onPressMessage: () => {},
  };

  renderMessageItem = ({ item }) => {
    // You'll add conditional rendering here for text, image, location
  };

  render() {
    const { messages } = this.props;
    return (
      <FlatList
        style={styles.container}
        inverted
        data={messages}
        renderItem={this.renderMessageItem}
        keyExtractor={keyExtractor}
        keyboardShouldPersistTaps={'handled'}
      />
    );
  }
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
```

```
    overflow: 'visible', // Prevents clipping on resize!
  },
});
```

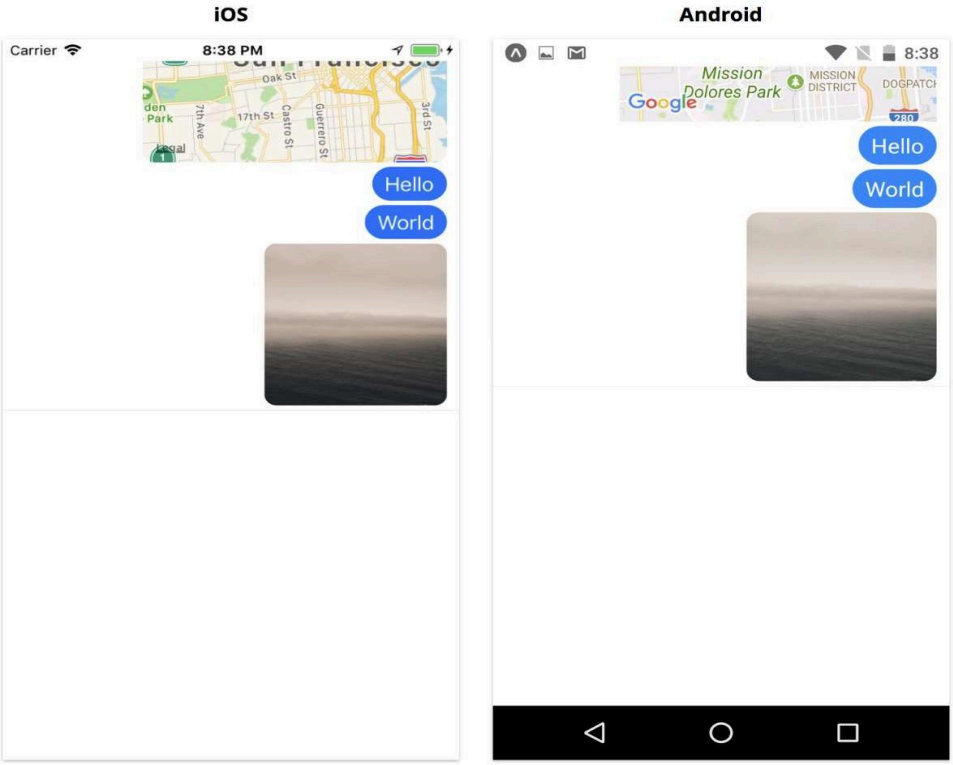
We looked at the data, renderitem, and keyExtractor props in the previous chapter. There are a few new props here that are worth looking at in more detail.

Question: Which are the new props introduced in the code above? Include them in the table below.

Prop Name	Description	Syntax

<u>Rendering Messages</u>

We've successfully set up a scrolling list, so now we can populate it with messages. As a reminder, this is what we're aiming to build:



Let's start with the styles. Conceptually, each message is a row in the list, so let's call our top-level message style messageRow and give it flexDirection: 'row'. We want to align messages to the right using justifyContent: 'flex-end', but leave a little space on the left with marginLeft: 60 in case our

message gets long. Text messages should appear in blue bubbles: this is what messageBubble is for.

Include your code for the indicated style (along with your own modifications below). Make sure to test the output so you have a screenshot of the app and the code.

Code Screenshot
App View

Let's move on to renderMessageItem and begin using the styles we just created. We can start by updating our imports to include all of the components we'll render from MessageList:

```javascript
import { FlatList, Image, StyleSheet, Text, TouchableOpacity, View } from 'react-native';
import { MapView } from 'expo';
```

Here we'll use MapView for the first time. You'll notice we import this from Expo, rather than from React Native. MapView comes from the 3rd party module react-native-maps, which Expo includes by default.

Add the following to your MessageList.js:

messaging/components/MessageList.js

```javascript
renderMessageItem = ({ item }) => {
  const { onPressMessage } = this.props;
  return (
    <View key={item.id} style={[styles.messageRow]}>
      <TouchableOpacity onPress={() => onPressMessage(item)}>
        {this.renderMessageBody(item)}
      </TouchableOpacity>
    </View>
  );
};

renderMessageBody = ({ type, text, uri, coordinate }) => {
  // You'll add conditional rendering here for text, image, location
};
```

How is react able to keep track of each item in the list here? Provide your observations.

Lastly, we call this.renderMessageBody with the item (our message), where we'll render the body of

each message. We'll switch on the type of the message to decide what to render.

messaging/components/MessageList.js

```javascript
renderMessageBody = ({ type, text, uri, coordinate }) => {
  switch (type) {
    case 'text':
      return (
        <View style={styles.messageBubble}>
          <Text style={styles.text}>{text}</Text>
        </View>
      );

    case 'image':
      return <Image style={styles.image} source={{ uri }} />;

    case 'location':
      return (
        <MapView
          style={styles.map}
          initialRegion={{
            ...coordinate,
            latitudeDelta: 0.08,
            longitudeDelta: 0.04,
          }}
        >
          <MapView.Marker coordinate={coordinate} />
        </MapView>
      );

    default:
      return null;
  }
};
```

Each type of message, text, image, and location has a different kind of UI. Most of the components we used for text and image should look pretty familiar.

Describe or demonstrate the UI for each kind of input (if applicable). Present screenshots below.

Add Message List to App.js

To test our new MessageList component, we can render it from within the renderMessageList method of App.js. Heading back to App.js now, we'll need to import our new Message List component and our utility functions for creating messages:

messaging/App.js

```javascript
// ...
import MessageList from './components/MessageList';
import { createImageMessage, createLocationMessage, createTextMessage } from
'./utils/MessageUtils';
// ...
```

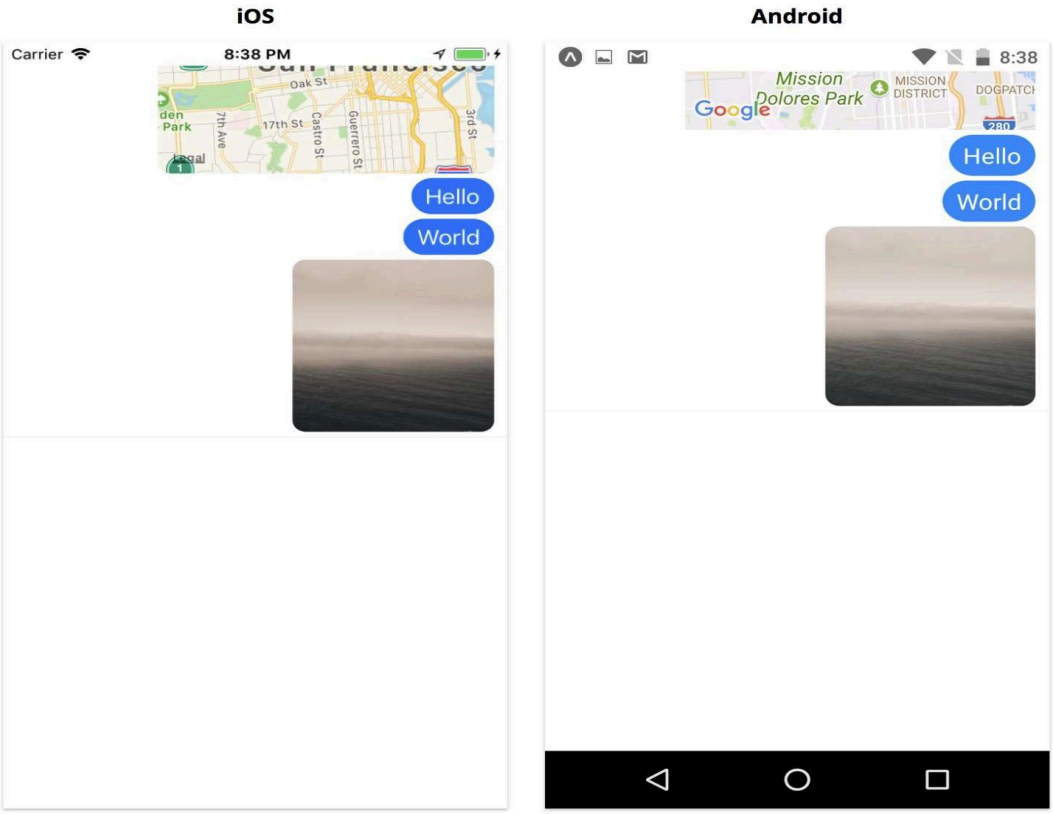
We can use these utility functions to create a few sample messages in the initial state of our app:

messaging/App.js

```javascript
state = {
  messages: [
    createImageMessage('https://unsplash.it/300/300'),
    createTextMessage('World'),
    createTextMessage('Hello'),
    createLocationMessage({
      latitude: 37.78825,
      longitude: -122.4324,
    }),
  ],
};

handlePressMessage = () => {}

renderMessageList() {
  const { messages } = this.state;

  return (
    <View style={styles.content}>
      <MessageList
        messages={messages}
        onPressMessage={this.handlePressMessage}
```

```
      />
    </View>
  );
}
```

We've now hooked up the hard coded message data with our MessageList component. We also added a placeholder handlePressMessage for handling tapping messages. When you save App.js, if everything is working correctly, here's what you should see:



Run your app and view your output, is it similar to what is shown above? Provide a screenshot.

Screenshot
Explanation

If not, what steps did you take to troubleshoot the code?

We're successfully rendering our message list! We can display messages of different types and our new messages appear at the bottom, just like we'd expect from a messaging app. At this point, we have the

prop onPressMessages set to the empty function handlePressMessage. Let's hook up a few different actions to onPressMessages.

5. Output

The Message Utils

- **Code**

JavaScript

```javascript
//MessageList.js
import PropTypes from 'prop-types';

// Define the shape for validation
export const MessageShape = PropTypes.shape({
  id: PropTypes.number.isRequired,
  type: PropTypes.oneOf(['text', 'image', 'location']),
  text: PropTypes.string,
  uri: PropTypes.string,
  coordinate: PropTypes.shape({
    latitude: PropTypes.number.isRequired,
    longitude: PropTypes.number.isRequired,
  }),
});

let messageId = 0;

// Helper to ensure unique IDs for FlatList keys
function getNextId() {
  messageId += 1;
  return messageId;
}

export function createTextMessage(text) {
  return {
    type: 'text',
    id: getNextId(),
    text,
  };
}

export function createImageMessage(uri) {
  return {
    type: 'image',
    id: getNextId(),
    uri,
  };
```

```
}

export function createLocationMessage(coordinate) {
  return {
    type: 'location',
    id: getNextId(),
    coordinate,
  };
}
```

- **Question**
  - **What is the benefit of declaring models this way?**
    - Declaring models using PropTypes.shape serves as a contract for your data structures. The primary benefits are: React will automatically warn you in the console if a component receives data that doesn't match the defined shape, such as a missing ID or an incorrect data type. It acts as clear documentation for other developers (or your future self) to understand exactly what properties a "message" object contains without having to hunt through the logic. It helps catch "undefined" errors early in the development process by ensuring that required fields, like the message ID, are always present.
  - **Why is it important for us to ensure the uniqueness of each id?**
    - To ensure each ID is unique because these IDs will be used as keys when rendering messages in a FlatList. Unique keys allow React to track which items have changed, been added, or been removed, which optimizes rendering performance and maintains the internal state of list items.

<u>MessageList</u>

```javascript
import React from 'react';
import PropTypes from 'prop-types';
import { FlatList, StyleSheet, View, TouchableOpacity, Text, Image } from 'react-native';
import { MessageShape } from '../utils/MessageUtils';

const keyExtractor = item => item.id.toString();

export default class MessageList extends React.Component {
  static propTypes = {
    messages: PropTypes.arrayOf(MessageShape).isRequired,
    onPressMessage: PropTypes.func,
  };
```

```
  static defaultProps = {
    onPressMessage: () => {},
  };

  renderMessageItem = ({ item }) => {
    const { onPressMessage } = this.props;

    return (
      <View key={item.id} style={styles.messageRow}>
        <TouchableOpacity onPress={() => onPressMessage(item)}>
          {this.renderMessageBody(item)}
        </TouchableOpacity>
      </View>
    );
  };

  renderMessageBody = ({ type, text, uri, coordinate }) => {
    return null;
  };

  render() {
    const { messages } = this.props;

    return (
      <FlatList
        style={styles.container}
        inverted
        data={messages}
        renderItem={this.renderMessageItem}
        keyExtractor={keyExtractor}
        keyboardShouldPersistTaps={'handled'}
      />
    );
  }
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    overflow: 'visible',
  },
  messageRow: {
```

```
    flexDirection: 'row',
    justifyContent: 'flex-end',
    marginLeft: 60,
  },
});
```

Question: Which are the new props introduced in the code above? Include them in the table below.

Prop Name	Description	Syntax
inverted	Reverses the scroll direction so the list starts at the bottom.	inverted
keyboardShouldPersistTaps	Determines if the keyboard stays up when tapping the list.	keyboardShouldPersistTaps={'handled'}

<u>Rendering Messages</u>

Code Screenshot

```javascript
// App.js
import React from 'react';
import { StyleSheet, View } from 'react-native';
import MessageList from './components/MessageList';
import { createImageMessage, createLocationMessage, createTextMessage } from './utils/MessageUtils';

export default class App extends React.Component {
  state = {
    messages: [
      createImageMessage('https://unsplash.it/300/300'),
      createTextMessage('World'),
      createTextMessage('Hello'),
      createLocationMessage({
        latitude: 37.78825,
        longitude: -122.4324,
      }),
    ],
  };

  handlePressMessage = () => {
```

```
    // Placeholder for now
  };

  renderMessageList() {
    const { messages } = this.state;
    return (
      <View style={styles.content}>
        <MessageList
          messages={messages}
          onPressMessage={this.handlePressMessage}
        />
      </View>
    );
  }

  render() {
    return (
      <View style={styles.container}>
        {this.renderMessageList()}
      </View>
    );
  }
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: 'white',
  },
  content: {
    flex: 1,
    backgroundColor: 'white',
  },
});
```

JavaScript

```
// MessageUtils.js
import PropTypes from 'prop-types';

export const MessageShape = PropTypes.shape({
  id: PropTypes.number.isRequired,
  type: PropTypes.oneOf(['text', 'image', 'location']),
  text: PropTypes.string,
  uri: PropTypes.string,
  coordinate: PropTypes.shape({
    latitude: PropTypes.number.isRequired,
    longitude: PropTypes.number.isRequired,
  }),
});

let messageId = 0;

function getNextId() {
```

```
  messageId += 1;
  return messageId;
}

export function createTextMessage(text) {
  return {
    type: 'text',
    id: getNextId(),
    text,
  };
}

export function createImageMessage(uri) {
  return {
    type: 'image',
    id: getNextId(),
    uri,
  };
}

export function createLocationMessage(coordinate) {
  return {
    type: 'location',
    id: getNextId(),
    coordinate,
  };
}
```

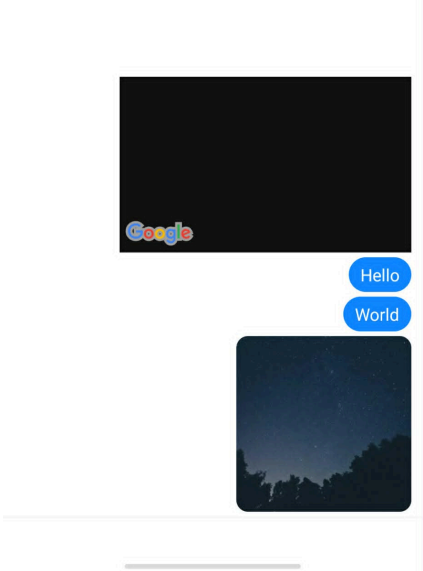
JavaScript

```
// MessageList.js
import React from 'react';
import PropTypes from 'prop-types';
import { FlatList, StyleSheet, Image, Text, TouchableOpacity, View } from 'react-native';
import MapView, { Marker } from 'react-native-maps';
import { MessageShape } from '../utils/MessageUtils';

const keyExtractor = item => item.id.toString();

export default class MessageList extends React.Component {
  static propTypes = {
    messages: PropTypes.arrayOf(MessageShape).isRequired,
    onPressMessage: PropTypes.func,
  };

  static defaultProps = {
    onPressMessage: () => {},
  };

  renderMessageItem = ({ item }) => {
    const { onPressMessage } = this.props;
    return (
      <View key={item.id} style={styles.messageRow}>
```

```
        <TouchableOpacity onPress={() => onPressMessage(item)}>
          {this.renderMessageBody(item)}
        </TouchableOpacity>
      </View>
    );
  };

  renderMessageBody = ({ type, text, uri, coordinate }) => {
    switch (type) {
      case 'text':
        return (
          <View style={styles.messageBubble}>
            <Text style={styles.text}>{text}</Text>
          </View>
        );
      case 'image':
        return <Image style={styles.image} source={{ uri }} />;
      case 'location':
        return (
          <MapView
            style={styles.map}
            initialRegion={{
              ...coordinate,
              latitudeDelta: 0.08,
              longitudeDelta: 0.04,
            }}
          >
            <Marker coordinate={coordinate} />
          </MapView>
        );
      default:
        return null;
    }
  };

  render() {
    const { messages } = this.props;
    return (
      <FlatList
        style={styles.container}
        inverted
        data={messages}
        renderItem={this.renderMessageItem}
        keyExtractor={keyExtractor}
        keyboardShouldPersistTaps={'handled'}
      />
    );
  }
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    overflow: 'visible',
  },
  messageRow: {
    flexDirection: 'row',
```

```
    justifyContent: 'flex-end',
    marginBottom: 4,
    marginRight: 10,
    marginLeft: 60,
  },
  messageBubble: {
    paddingVertical: 5,
    paddingHorizontal: 10,
    backgroundColor: 'rgb(16,135,255)',
    borderRadius: 20,
  },
  text: {
    color: 'white',
    fontSize: 16,
  },
  image: {
    width: 150,
    height: 150,
    borderRadius: 10,
  },
  map: {
    width: 250,
    height: 250,
    borderRadius: 10,
  },
});
```

App View



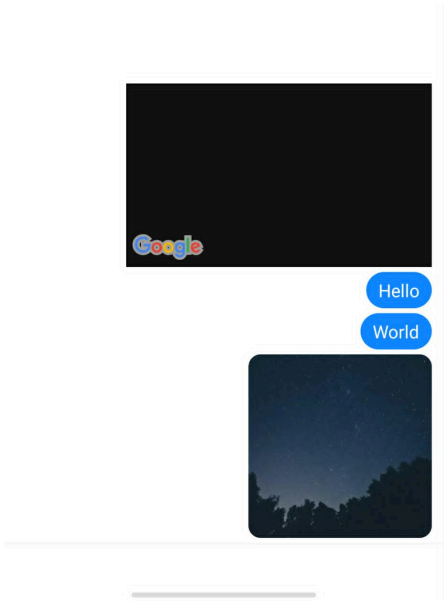
- Currently this is the output, but they are not clickable for now

- **How is react able to keep track of each item in the list here? Provide your observations.**

- React is able to keep track of each item in the list by utilizing the keyExtractor prop within the FlatList component. In our implementation, this is defined as const keyExtractor = item => item.id.toString();, which assigns a stable, unique identity to every message based on its ID. My observation is that this unique key allows React to perform efficient "reconciliation," meaning it can identify exactly which message has been added, removed, or changed without having to re-render the entire list, which significantly improves app performance.

- **Describe or demonstrate the UI for each kind of input (if applicable). Present screenshots below.**
  - Each message type in the application is rendered with a distinct user interface tailored to its data type:
    - **Text Messages:** These are rendered as blue rounded bubbles (messageBubble) containing white text. They are aligned to the right side of the screen to distinguish them as outgoing messages.
    - **Image Messages:** These use the Image component to display a graphic thumbnail based on a provided URL. The UI displays the image with a fixed size and rounded corners to maintain consistency with the chat theme.
    - **Location Messages:** These utilize an interactive MapView component that displays a specific geographic region. The UI includes a Marker (pin) at the exact coordinates provided, allowing the user to see a visual map preview within the chat feed.

Add Message List to App.js

Screenshot

Explanation
● The FlatList in MessageList.js uses a renderItem function to iterate through the messages array

stored in the App.js state.

- The inverted prop is applied to the FlatList, which causes the scroll view to start from the bottom. This ensures that new messages appear at the bottom of the screen, mimicking a standard chat application.
- The renderMessageBody function performs a switch on the message type to determine the visual output:
  - **Text**: Renders a blue messageBubble containing a Text component.
  - **Image**: Renders an Image component using the provided uri.
  - **Location**: Renders a MapView focused on specific coordinates with a MapView.Marker pinned at that location
- The messageRow style uses flexDirection: 'row' and justifyContent: 'flex-end', which pushes all message content to the right side of the screen.
- The MessageShape defined in the utilities ensures that each rendered object contains a unique id and a valid type, which allows the keyExtractor to track items efficiently during scrolls or updates.

6. Supplementary Activity

The following activities contribute to the attainment of ILO1 and ILO2 for this lab activity.

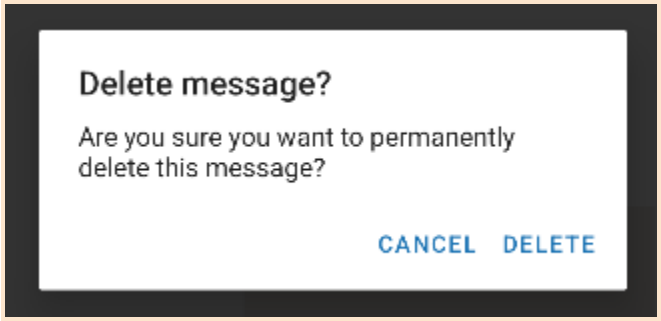
Modify the Messaging application to add the following functionalities using appropriate APIs in React Native.

a. **Alert.alert API.**
b. Fullscreen Image with Backhandler.

a. Alert.alert API.

The first action we'll add is to text messages. We'll add a "delete" feature: when the user taps a text message, we'll give the user the option to delete that message. We'll present a dialog with two choices: delete and cancel. We can use the Alert.alert API to present the user with a native dialog window for making this choice. Provide code and screenshots showing that the user is able to delete a message.

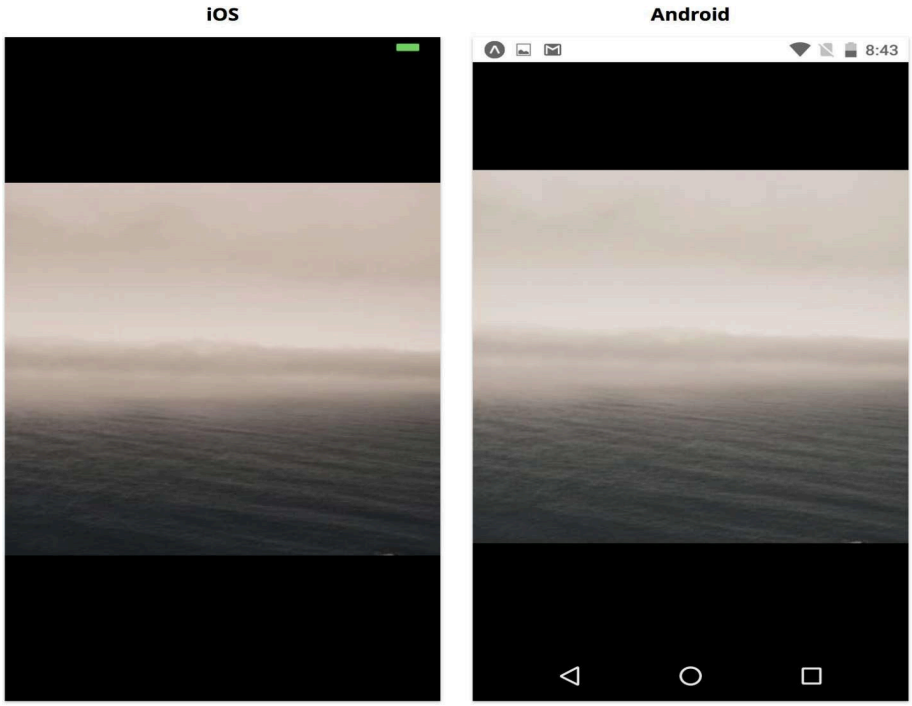
```
handlePressMessage = ({ id, type, text }) => {
  switch (type) {
    case 'text':
      Alert.alert(
        'Delete message?',
        'Are you sure you want to permanently delete this message?',
        [
          { text: 'Cancel', style: 'cancel' },
          {
            text: 'Delete',
            style: 'destructive',
            onPress: () => {
              const { messages } = this.state;
              this.setState({
                messages: messages.filter(message => message.id !== id),
              });
            },
          },
        ],
      );
      break;
    case 'image':
      this.setState({ fullscreenImageId: id });
      break;
    default:
      break;
  }
};
```



- When the user presses a message of the type 'text', the handlePressMessage function is executed. The Alert.alert API immediately displays a native dialog window. This dialog contains the title "Delete message?" and the inquiry "Are you sure you want to permanently delete this message?
- **User Choice**: The user is presented with two clear options: **"Cancel"** and **"Delete"**.
  - Cancel: Dismisses the dialog without making changes.
  - Delete: Executes an onPress callback that filters the messages array in the application state to remove the message with that specific id, effectively deleting it from the UI.

b. Fullscreen Image with Backhandler for Android

When the user presses an image, we'll show it fullscreen. Sample output:



Transitioning to full screen might be accomplished using a navigation library (which will be covered in future activities), but we can also do it manually. If we do, we'll want the Android back button to dismiss the fullscreen image - we can use the BackHandler API to accomplish this. Let's also dismiss the image when it's pressed again, so that we're not trapped in a fullscreen image state on IOS. In App.js, we'll use state to keep track of which image was pressed. Let's add the following for state tracking:

messaging/App.js

```javascript
// ...
state = {
    // …
    fullscreenImageId: null,
};

// …

dismissFullscreenImage = () => {
    this.setState({ fullscreenImageId: null });
};
```

```
// …
```

We've initialized fullscreen mageId to null to indicate that we don't want to show any image. Then, when a message is pressed, we'll set it to the id of the message object. We can update handlePressMessage with the following:

messaging/App.js

```javascript
// ...
handlePressMessage = ({ id, type }) => {
  switch (type) {
    case 'text':
      // ...
    case 'image':
      this.setState({ fullscreenImageId: id });
      break;
    default:
      break;
  }
};
// …
```

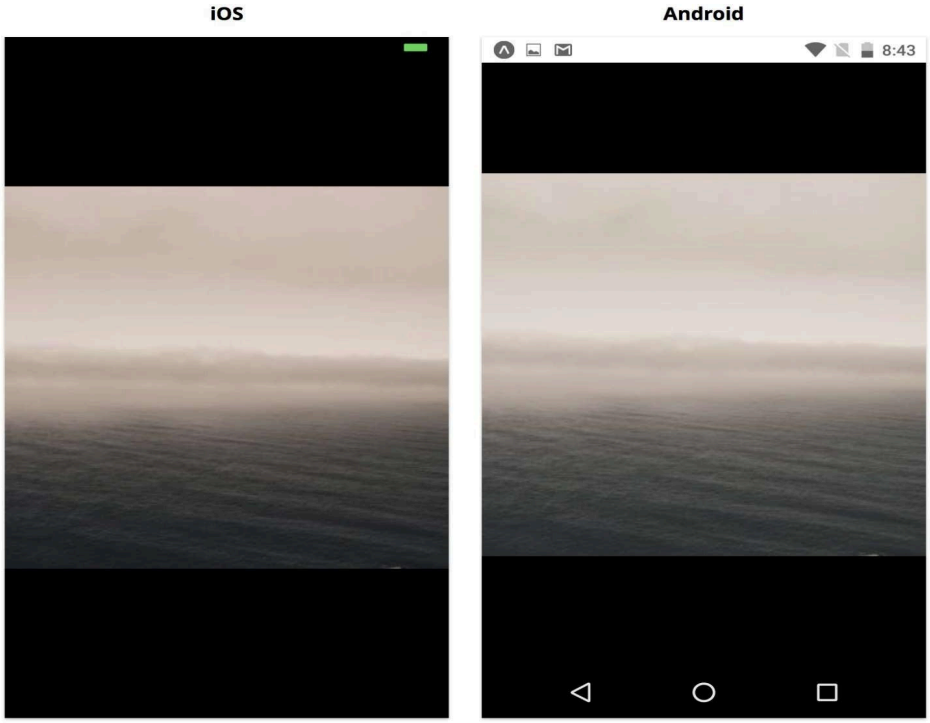
Now we need to render the fullscreen image. We'll use a Touchable Highlight for the black overlay background so we can tap it to dismiss the image. Within that, we'll use an Image. We'll use the fullscreen mageId to look up which image we need to display as the uri of the Image component.

Add the following to messaging/App.js

```javascript
// ...
renderFullscreenImage = () => {
  const { messages, fullscreenImageId } = this.state;
  if (!fullscreenImageId) return null;
  const image = messages.find(message => message.id === fullscreenImageId);
  if (!image) return null;
  const { uri } = image;
  return (
    <TouchableHighlight style={styles.fullscreenOverlay}
```

```
      onPress={this.dismissFullscreenImage}>
      <Image style={styles.fullscreenImage} source={{ uri }} />
    </TouchableHighlight>
  );
};
// ...
render() {
  return (
    <View style={styles.container}>
      <Status />
      {this.renderMessageList()}
      {this.renderToolbar()}
      {this.renderInputMethodEditor()}
      {this.renderFullscreenImage()}
    </View>
  );
}
```

Save App.js. Now when we tap an image, we should see it fullscreen, and we can tap it again to dismiss it:



On Android though, we'll also want the device's back button to dismiss the image. We do this using the BackHandler API.

Provide a screenshot of your application being able to view an image in fullscreen. If you had to troubleshoot the code, indicate below.

```
case 'image':
  this.setState({ fullscreenImageId: id });
  break;
default:
  break;
```

- The app uses fullscreenImageId in the state to manually toggle between the message list and a dedicated image overlay without needing a navigation library. By placing {this.renderFullscreenImage()} at the end of the render function, the image is drawn last, ensuring it appears on the top-most layer and covers all other UI elements. The BackHandler API intercepts the physical Android back button, allowing users to dismiss the enlarged image using standard device navigation. Event listeners are carefully managed in componentWillMount and componentWillUnmount to ensure the back button logic only runs when the component is active, preventing memory leaks. The combination of TouchableHighlight and BackHandler creates a consistent "tap-to-dismiss" or "back-to-dismiss" experience for both iOS and Android users.
- **TAKE NOTE: If you notice that the image is different from the previous its because I've switch from expo go android to android studio to screenshot the output easily**

<u>Backhandler API</u>

Like with NetInfo, we use the event listener pattern to handle back button press events:

```javascript
BackHandler.addEventListener('hardwareBackPress', handlerFunction);
```

We can use this to get notified every time the user presses the back button on an Android device. We'll have our handlerFunction hide our full screen image.

Step 1: First, import the BackHandler API: messaging/App.js

```javascript
import {
    // other imports
    BackHandler,
} from 'react-native';
```

Step 2: Then we'll use componentWillMount and componentWill Unmount to listen to back button presses:
messaging/App.js

```javascript
// ...
componentWillMount() {
    this.subscription = BackHandler.addEventListener('hardwareBackPress', () => {
        const { fullscreenImageId } = this.state;
        if (fullscreenImageId) {
            this.dismissFullscreenImage();
            return true;
        }
        return false;
    });
}

componentWillUnmount() {
    this.subscription.remove();
}
// …
```

Step 3: Demonstrate

Our message list is working pretty well now! We've used two core APIs, Alert and BackHandler, to create cross-platform interactions for deleting and enlarging messages.

- The BackHandler API allows the app to detect and respond to the physical "Back" button press on Android devices. It uses an event listener established in componentWillMount to notify the app every time the hardware button is pressed. The handler is programmed to only intercept the event if an image is currently fullscreen; otherwise, it returns false to allow

the default system behavior. The listener is specifically removed in componentWillUnmount to ensure the app doesn't waste resources or cause errors after the component is destroyed. This creates a seamless "cross-platform interaction" where Android users can dismiss enlarged images using native hardware controls rather than just on-screen taps.

JavaScript

```javascript
// App.js
import React from 'react';
import { StyleSheet, View, Alert, TouchableHighlight, Image, BackHandler } from 'react-native';

// Import custom components
import Status from './components/Status';
import MessageList from './components/MessageList';

// Import utilities
import {
  createImageMessage,
  createLocationMessage,
  createTextMessage
} from './utils/MessageUtils';

export default class App extends React.Component {
  state = {
    messages: [
      createImageMessage('https://unsplash.it/300/300'),
      createTextMessage('Hello'),
      createTextMessage('World'),
      createLocationMessage({
        latitude: 37.78825,
        longitude: -122.4324,
      }),
    ],
    fullscreenImageId: null,
  };

  componentDidMount() {
    this.subscription = BackHandler.addEventListener('hardwareBackPress', () => {
      const { fullscreenImageId } = this.state;
      if (fullscreenImageId) {
        this.dismissFullscreenImage();
        return true;
      }
      return false;
    });
```

```
}

componentWillUnmount() {
  this.subscription.remove();
}

dismissFullscreenImage = () => {
  this.setState({ fullscreenImageId: null });
};

handlePressMessage = ({ id, type, text }) => {
  switch (type) {
    case 'text':
      Alert.alert(
        'Delete message?',
        'Are you sure you want to permanently delete this message?',
        [
          { text: 'Cancel', style: 'cancel' },
          {
            text: 'Delete',
            style: 'destructive',
            onPress: () => {
              const { messages } = this.state;
              this.setState({
                messages: messages.filter(message => message.id !== id),
              });
            },
          },
        ],
      );
      break;
    case 'image':
      this.setState({ fullscreenImageId: id });
      break;
    default:
      break;
  }
};

renderFullscreenImage = () => {
  const { messages, fullscreenImageId } = this.state;
  if (!fullscreenImageId) return null;
```

```
    const image = messages.find(m => m.id === fullscreenImageId);
    if (!image) return null;

    const { uri } = image;

    return (
      <TouchableHighlight
        style={styles.fullscreenOverlay}
        onPress={this.dismissFullscreenImage}
      >
        <Image style={styles.fullscreenImage} source={{ uri }} />
      </TouchableHighlight>
    );
  };

  renderMessageList() {
    const { messages } = this.state;
    return (
      <View style={styles.content}>
        <MessageList
          messages={messages}
          onPressMessage={this.handlePressMessage}
        />
      </View>
    );
  }

  render() {
    return (
      <View style={styles.container}>
        <Status />
        {this.renderMessageList()}
        <View style={styles.toolbar}></View>
        <View style={styles.inputMethodEditor}></View>
        {this.renderFullscreenImage()}
      </View>
    );
  }
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
```

```
    backgroundColor: 'white',
  },
  content: {
    flex: 1,
    backgroundColor: 'white',
  },
  toolbar: {
    borderTopWidth: 1,
    borderTopColor: 'rgba(0,0,0,0.04)',
    backgroundColor: 'white',
    height: 50,
  },
  inputMethodEditor: {
    flex: 0,
    backgroundColor: 'white',
  },
  fullscreenOverlay: {
    ...StyleSheet.absoluteFillObject,
    backgroundColor: 'black',
    zIndex: 2,
    justifyContent: 'center',
    alignItems: 'center',
  },
  fullscreenImage: {
    width: '100%',
    height: '100%',
    resizeMode: 'contain',
  },
});
```

JavaScript

```
// MessageUtils.js
import PropTypes from 'prop-types';

// Define the shape for validation
export const MessageShape = PropTypes.shape({
  id: PropTypes.number.isRequired,
  type: PropTypes.oneOf(['text', 'image', 'location']),
  text: PropTypes.string,
  uri: PropTypes.string,
  coordinate: PropTypes.shape({
```

```
    latitude: PropTypes.number.isRequired,
    longitude: PropTypes.number.isRequired,
  }),
});

let messageId = 0;

// Helper to ensure unique IDs for FlatList keys
function getNextId() {
  messageId += 1;
  return messageId;
}

export function createTextMessage(text) {
  return {
    type: 'text',
    id: getNextId(),
    text,
  };
}

export function createImageMessage(uri) {
  return {
    type: 'image',
    id: getNextId(),
    uri,
  };
}

export function createLocationMessage(coordinate) {
  return {
    type: 'location',
    id: getNextId(),
    coordinate,
  };
}
```

JavaScript

```
// MessageList.js
import React from 'react';
```

```
import PropTypes from 'prop-types';
import { FlatList, StyleSheet, View, TouchableOpacity, Text, Image } from 'react-native';
import MapView, { Marker } from 'react-native-maps';
import { MessageShape } from '../utils/MessageUtils';

const keyExtractor = item => item.id.toString();

export default class MessageList extends React.Component {
  static propTypes = {
    messages: PropTypes.arrayOf(MessageShape).isRequired,
    onPressMessage: PropTypes.func,
  };

  static defaultProps = {
    onPressMessage: () => {},
  };

  renderMessageItem = ({ item }) => {
    const { onPressMessage } = this.props;

    return (
      <View key={item.id} style={styles.messageRow}>
        <TouchableOpacity onPress={() => onPressMessage(item)}>
          {this.renderMessageBody(item)}
        </TouchableOpacity>
      </View>
    );
  };

  renderMessageBody = ({ type, text, uri, coordinate }) => {
    switch (type) {
      case 'text':
        return (
          <View style={styles.messageBubble}>
            <Text style={styles.text}>{text}</Text>
          </View>
        );
      case 'image':
        return <Image style={styles.image} source={{ uri }} />;
      case 'location':
        return (
          <MapView
            style={styles.map}
```

```
          initialRegion={{
            ...coordinate,
            latitudeDelta: 0.08,
            longitudeDelta: 0.04,
          }}
        >
          {/* Use the named Marker component here */}
          <Marker coordinate={coordinate} />
        </MapView>
      );
    default:
      return null;
  }
};

render() {
  const { messages } = this.props;

  return (
    <FlatList
      style={styles.container}
      inverted
      data={messages}
      renderItem={this.renderMessageItem}
      keyExtractor={keyExtractor}
      keyboardShouldPersistTaps={'handled'}
    />
  );
}
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    overflow: 'visible',
  },
  messageRow: {
    flexDirection: 'row',
    justifyContent: 'flex-end',
    marginBottom: 4,
    marginRight: 10,
    marginLeft: 60,
  },
```

```
  messageBubble: {
    paddingVertical: 5,
    paddingHorizontal: 10,
    backgroundColor: 'rgb(16,135,255)',
    borderRadius: 20,
  },
  text: {
    color: 'white',
    fontSize: 16,
  },
  image: {
    width: 150,
    height: 150,
    borderRadius: 10,
  },
  map: {
    width: 250,
    height: 150,
    borderRadius: 10,
  },
});
```

7. Conclusion

- **A concise summary of the activity performed.**

In this activity, I developed a messaging application that utilizes React Native's core APIs to handle various data types, including text, images, and geographic locations. I implemented a MessageList component using a FlatList with an inverted prop to mimic a real chat interface where new messages appear at the bottom. Then , I integrated the Alert API to provide native dialogs for deleting messages and the BackHandler API to manage hardware back-button interactions for a fullscreen image mode.

- **Your personal conclusions / reflections.**

This activity demonstrated that building an advance mobile interface requires a deep understanding of how React Native bridges JavaScript logic with native device features. I realized that managing application state, such as tracking a fullscreenImageId, is essential for creating dynamic and interactive user experiences. Therefore, in the process I found that small details like using a unique keyExtractor are what truly ensure the performance and stability of a mobile application during complex data updates.

● **An overview of the lessons you learned from this module.** I learned how to use PropTypes to define data shapes, which serves as a vital debugging tool for ensuring component data integrity. I gained practical experience with the BackHandler API, specifically learning how to use the event listener pattern to intercept hardware-specific events on Android. Overall, I now have the knowledge to use of conditional rendering within a FlatList to display different UI components such as MapView or Image based on the specific type of data received.
8. Assessment Rubrics